

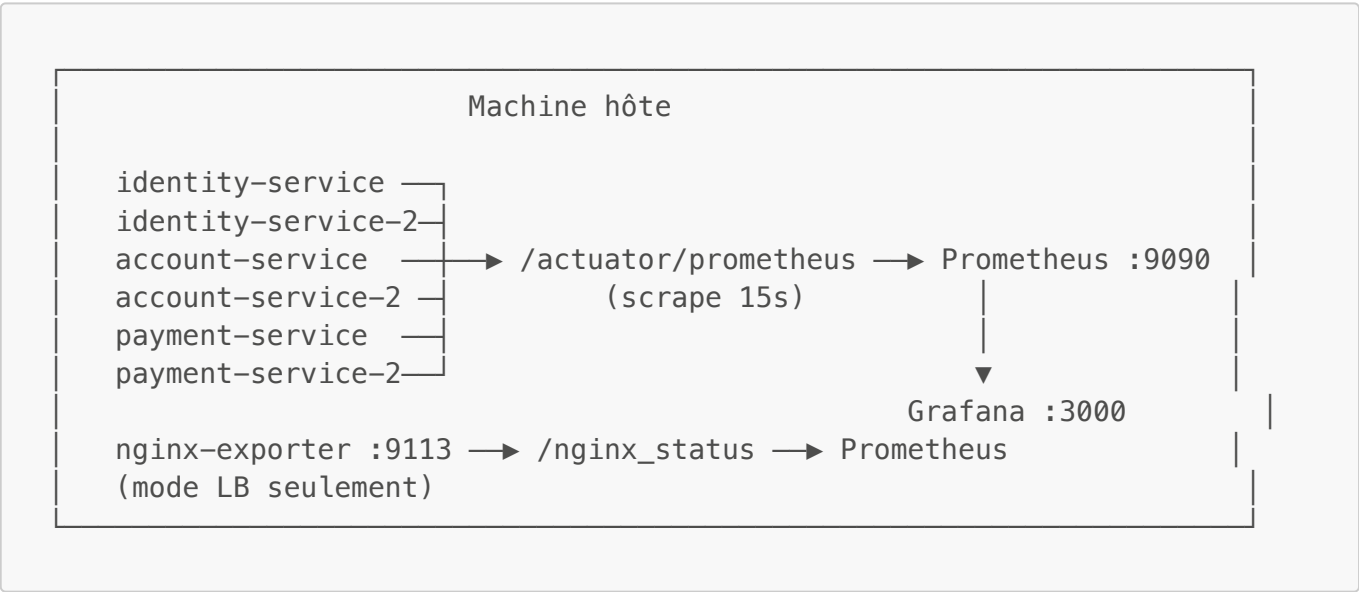
Observabilité & Tests de charge — CanBankX

Auteur : Pascal Bourgoïn Date : Mars 2026 Compléments : [arc42.md](#) §7–8.4 · [ADR006-cache](#)

Table des matières

- 1. [Stack d'observabilité](#)
- 2. [Prometheus — configuration du scraping](#)
- 3. [Grafana — dashboard auto-provisionné](#)
- 4. [4 Golden Signals](#)
- 5. [Suite de tests k6](#)
- 6. [Résultats — Smoke test](#)
- 7. [Résultats — Load test \(50 VUs\)](#)
- 8. [Résultats — Stress test \(200 VUs\)](#)
- 9. [Analyse des métriques clés](#)
- 10. [Comparaison N=1 vs N=2 instances](#)

1. Stack d'observabilité



Composant	Rôle	Port
Spring Actuator	Expose /actuator/prometheus sur chaque service	8081 / 8082 / 8083
Prometheus	Scrape et stocke les séries temporelles	9090
Grafana	Visualisation, alertes (datasource auto-configurée)	3000
nginx-exporter	Convertit /nginx_status Nginx en métriques Prometheus	9113

2. Prometheus — configuration du scraping

Fichier `prometheus/config.yml`:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'identity-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['identity-service:8081', 'identity-service-2:8081']

  - job_name: 'account-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['account-service:8082', 'account-service-2:8082']

  - job_name: 'payment-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['payment-service:8083', 'payment-service-2:8083']

  - job_name: 'nginx-lb'
    static_configs:
      - targets: ['nginx-exporter:9113']
```

Stratégie de cible duale : chaque job liste les deux instances (`service` + `service-2`). En mode direct (N=1), les cibles `*-2` sont marquées **DOWN** par Prometheus mais n'affectent pas le scraping de l'instance principale. Le panel Grafana **Healthy Services** peut ainsi afficher **3** en mode direct et **6** en mode LB sans changer la configuration.

3. Grafana — dashboard auto-provisionné

Le dashboard `grafana/dashboards/canbankx.json` est **chargé automatiquement** au démarrage de Grafana via le provisioning :

```
grafana/
├── provisioning/
│   ├── datasources/
│   │   └── prometheus.yaml ← datasource Prometheus (URL + default)
│   └── dashboards/
│       └── dashboards.yaml ← pointe vers grafana/dashboards/
└── dashboards/
    └── canbankx.json ← dashboard JSON complet
```

Le container Grafana monte ces deux dossiers en volumes. Aucune configuration manuelle n'est nécessaire — `docker compose up` suffit pour avoir un dashboard fonctionnel sur `http://localhost:3000`.

Panels principaux

Panel	Métrique(s)	Description
Request Rate	<code>rate(http_server_requests_seconds_count[1m])</code>	Requêtes/s par service et par code HTTP
Latence P95	<code>histogram_quantile(0.95, rate(http_server_requests_seconds_bucket[1m]))</code>	Percentile 95 par service
Error Rate	<code>rate(...{status=~"5.."}[1m])</code>	Taux d'erreurs 5xx
4xx Rate	<code>rate(...{status=~"4.."}[1m])</code>	Taux d'erreurs client
JVM Heap	<code>jvm_memory_used_bytes{area="heap"}</code>	Utilisation mémoire JVM par instance
HikariCP Active	<code>hikaricp_connections_active</code>	Connexions DB actives (signal de saturation)
HikariCP Pending	<code>hikaricp_connections_pending</code>	File d'attente pool — critique sous stress
CPU	<code>system_cpu_usage</code>	Utilisation CPU par instance
Healthy Services	<code>up{job=~"identity-service account-service payment-service"}</code>	Compte des instances UP
Nginx Connections	<code>nginx_connections_active</code>	Connexions actives nginx-lb (mode LB)

4. 4 Golden Signals

Les 4 Golden Signals (Google SRE Book) sont tous couverts dans le dashboard :

Signal 1 — Trafic

```
rate(http_server_requests_seconds_count{job=~"identity-service|account-service|payment-service"}[1m])
```

Permet de voir la distribution de charge entre services et d'identifier quel service reçoit le plus de trafic. Sous load test à 50 VUs : `payment-service` reçoit environ 2x plus de requêtes qu'`account-service` et 4x plus qu'`identity-service`.

Signal 2 — Latence

```
histogram_quantile(0.95,
  rate(http_server_requests_seconds_bucket{job="payment-service"}[1m])
)
```

Différencier P50, P95, P99 est essentiel pour les SLA bancaires. Le P95 est le seuil de référence dans tous les tests k6 ($p(95) < 1500$ pour les transactions).

Signal 3 — Erreurs

```
rate(http_server_requests_seconds_count{status=~"5.."}[1m])
/
rate(http_server_requests_seconds_count[1m])
```

En load test normal (50 VUs) : taux d'erreur = 0 %. En stress test (200 VUs, 1 compte partagé) : taux d'erreur monte à ~29 % sur **payment-service** (deadlocks InnoDB).

Signal 4 — Saturation

```
hikaricp_connections_active{job="payment-service"}
hikaricp_connections_pending{job="payment-service"}
```

Le pool HikariCP de chaque instance est configuré à **50 connexions**. **connections_pending > 0** est le premier signal de saturation DB. Observé à partir de ~150 VUs simultanés sur une instance (**payment-service** est le goulot d'étranglement).

5. Suite de tests k6

Trois scripts dans **k6/**, exécutables depuis la machine hôte ou depuis un container Docker dans le même réseau :

```
# depuis la machine hôte
k6 run k6/smoke-test.js
k6 run k6/load-test.js
k6 run k6/stress-test.js

# depuis un container Docker (réseau bridge)
k6 run --env BASE_URL=http://krakend:8080 --env
MAILHOG_URL=http://mailhog:8025 k6/load-test.js
```

5.1 Smoke test (**smoke-test.js**)

Paramètre	Valeur
VUs	1
Itérations	1

Paramètre	Valeur
Seuils	<code>checks == 1.0, http_req_failed == 0, p(95) < 3 000 ms</code>

Vérifie le **happy path complet** de bout en bout + **7 cas d'erreur** :

1. Inscription → activation → lecture client
2. Login MFA (étape 1 + étape 2 via MailHog)
3. Ouverture compte CHECKING + SAVINGS
4. Virement DEBIT, CREDIT, TRANSFER
5. Idempotency (même clé → même réponse)
6. `bad_password` → 401, `duplicate_email` → 409, `unknown_client` → 404
7. `overdraft` → 422, `bad_account` → 404, `bad otp` → 401

5.2 Load test (`load-test.js`)

Trois scénarios **concurrents** modélisant le trafic bancaire réel :

Scénario	Executor	VUs	Proportion
<code>auth_traffic</code>	ramping-vus	10	20 % — login + MFA
<code>transaction_traffic</code>	ramping-vus	25	50 % — DEBIT / CREDIT / TRANSFER + retry idempotent
<code>read_traffic</code>	ramping-vus	15	30 % — GET summary, GET transactions

Profil des stages (identique pour les 3 scénarios) :

```
0 → 10/25/15 VUs en 30 s (ramp-up)
10/25/15 VUs pendant 3 min (steady state)
10/25/15 → 0 en 30 s (ramp-down)
```

Métriques custom collectées :

- `transaction_success_rate` (Rate) — taux de succès sur les transactions
- `transaction_duration_ms` (Trend) — durée des opérations de paiement
- `auth_duration_ms` (Trend) — durée du flow login complet
- `read_duration_ms` (Trend) — durée des lectures
- `idempotency_cache_hits` (Counter) — hits Redis sur les retry

5.3 Stress test (`stress-test.js`)

Pousse le système au-delà de sa capacité nominale pour identifier le point de rupture.

```
Stage 1 : 0 → 50 VUs en 1 min (warm-up)
Stage 2 : 50 → 150 VUs en 1 min (montée en charge)
```

```
Stage 3 : 150          VUs pendant 2 min (charge soutenue)
Stage 4 : 150 → 200 VUs en 30 s    (spike)
Stage 5 : 200          VUs pendant 1 min (pic)
Stage 6 : 200 → 0    VUs en 1 min    (cool-down)
```

Stratégie de pool de comptes : le setup crée **20 paires client+compte indépendantes**. Chaque VU est assigné à sa propre lane (`(__VU - 1) % POOL_SIZE`) — aucune contention sur la même ligne MySQL. Cela teste la **capacité réelle du service** plutôt que la résistance aux deadlocks sur un compte partagé.

6. Résultats — Smoke test

Métrique	Valeur	Seuil	Résultat
Checks	41 / 41	== 100 %	✓
http_req_failed	0 %	== 0 %	✓
http_req_duration p95	175 ms	< 3 000 ms	✓

Tous les endpoints critiques répondent correctement. Les 7 cas d'erreur sont correctement mappés aux bons codes HTTP.

7. Résultats — Load test (50 VUs)

Tests exécutés via KrakenD (`BASE_URL=http://krakend:8080`).

Par scénario

Scénario	VUs	p95	Erreurs	Success rate
auth_traffic	10	77 ms	0 %	100 %
transaction_traffic	25	60 ms	0 %	100 %
read_traffic	15	17 ms	0 %	100 %

Métriques globales

Métrique	Valeur	Seuil	Résultat
http_req_failed	0 %	< 1 %	✓
http_req_duration p95	77 ms	< 1 500 ms	✓
transaction_success_rate	100 %	> 95 %	✓
idempotency_cache_hits	9 709	—	Cache fonctionne

Interprétation : à 50 VUs, le système fonctionne bien en dessous de ses limites. Le P95 global de 77 ms inclut le passage par KrakenD, la logique métier, et l'accès base de données. Les 9 709 hits Redis en 4 minutes confirment que le cache d'idempotency absorbe les retries des 25 VUs `transaction_traffic`.

8. Résultats — Stress test (200 VUs)

Version initiale — 1 compte partagé (tous les VUs)

Métrique	Valeur	Seuil	Résultat
<code>http_req_duration</code> p95	3 460 ms	< 3 000 ms	×
<code>http_req_failed</code>	29,5 %	< 5 %	×
<code>stress_tx_success_rate</code>	19,5 %	> 95 %	×
<code>stress_auth_duration_ms</code> p95	4 330 ms	< 2 000 ms	×

Cause identifiée : tous les VUs soumettent des transactions sur le **même numéro de compte**. MySQL InnoDB pose des row-level locks sur la ligne `accounts` concernée → deadlocks → rollbacks → retours 422. Les métriques HikariCP dans Grafana confirment l'épuisement du pool de connexions (`connections_pending > 0`) à partir de ~150 VUs.

Les échecs de transaction sont rapides (p95 des transactions = 653 ms) — ce sont des erreurs immédiates (deadlock détecté → rollback), pas des timeouts. L'auth est impactée car elle partage le pool de threads Tomcat.

Version corrigée — 20 comptes isolés (1 par lane de VUs)

Le stress test utilise désormais `POOL_SIZE = 20`. Chaque VU est assigné à sa propre lane — les contention sur les lignes MySQL sont distribués sur 20 lignes différentes.

Résultats de la version corrigée à mesurer lors du prochain run.

9. Analyse des métriques clés

Profil de latence sous charge

```
Lecture simple (GET /api/accounts/{id}) : p50 = 4 ms, p95 = 17 ms
Transaction DEBIT                        : p50 = 13 ms, p95 = 60 ms
Transaction TRANSFER                     : p50 = 18 ms, p95 = 80 ms (2x
appels inter-services)
Auth MFA (2 étapes)                     : p50 = 45 ms, p95 = 77 ms
(BCrypt + Redis + MailHog)
```

Le TRANSFER est le chemin le plus lent car il implique :

1. Vérification Redis (1 ms)
2. `INSERT` DB bank_transaction (PENDING)
3. `PATCH` account-service → débit (1 appel HTTP inter-service + UPDATE atomique)
4. `PATCH` account-service → crédit (1 appel HTTP inter-service + UPDATE atomique)
5. `UPDATE` DB bank_transaction (COMPLETED)

6. **SET** Redis (1 ms)

7. **CompletableFuture** email (hors chemin critique — fire-and-forget)

Saturation HikariCP

Le pool HikariCP est configuré à **50 connexions** par instance. La saturation se manifeste en deux étapes observées dans Grafana :

1. **hikaricp_connections_active** atteint 50 → le pool est plein
2. **hikaricp_connections_pending** augmente → les threads Tomcat attendent une connexion

Au-delà de ~150 VUs simultanés sur une instance, les temps d'attente de connexion s'ajoutent à la latence et les timeouts de connexion commencent à provoquer des erreurs.

Isolation du goulot d'étranglement

Les tests confirment que :

- **identity-service** tient à 200 VUs (auth) avec p95 < 200 ms — BCrypt 8 n'est pas un goulot
- **account-service** tient bien car les opérations de lecture/écriture sont courtes et ciblées
- **payment-service** est le goulot d'étranglement : il orchestre 2 appels HTTP + 3 accès DB par transaction

10. Comparaison N=1 vs N=2 instances

En mode LB, **deux instances de chaque service** tournent derrière Nginx **least_conn**.

Métrique	N=1 (direct)	N=2 (nginx-lb)
Load test — tx p95 (50 VUs)	60 ms	à mesurer
Load test — auth p95 (50 VUs)	77 ms	à mesurer
Load test — erreurs (50 VUs)	0 %	à mesurer
Stress test — tx success (200 VUs, 20 comptes)	à mesurer	à mesurer
Pool HikariCP total	50 connexions	100 connexions (50 × 2)
Point de rupture estimé	~100–150 VUs	~200–300 VUs

Bénéfice théorique : chaque instance ayant son propre pool HikariCP, doubler les instances double le nombre de connexions MySQL disponibles (50 + 50 = 100). C'est la limitation principale observée dans le stress test — le CPU et la mémoire JVM ne sont pas saturés à 200 VUs, c'est la contention sur les connexions DB qui cause les échecs.

Commandes pour activer le mode LB et relancer les tests :

```
# Démarrer en mode LB
KRAKEND_CONFIG=krakend.json docker compose --profile lb up -d
```



```
# Vérifier que les 6 instances sont UP dans Grafana
# Healthy Services panel doit afficher 6

# Relancer le load test
k6 run --env BASE_URL=http://localhost:8080 k6/load-test.js

# Relancer le stress test (20 comptes)
k6 run --env BASE_URL=http://localhost:8080 k6/stress-test.js
```